Security in Software Applications
Homework 1

Rando, 1985084

A.A. 2024/2025

1 Flawfinder

Flawfinder is a specialized static analysis tool designed to thoroughly examine source code written in C or C++. Unlike dynamic analysis tools that require code execution, Flawfinder operates by analyzing the code at rest, without compiling or running the application. Its primary function is to identify and report potential security vulnerabilities, often referred to as 'flaws,' that could expose a system to various forms of attack. It achieves this by scanning the codebase against a built-in database of C/C++ functions and patterns known to be associated with common security issues, such as buffer overflows, format string vulnerabilities, and race conditions. By highlighting these potential weaknesses early in the software development lifecycle, Flawfinder assists developers in proactively addressing security risks, thereby contributing to the creation of more robust and secure software.

1.1 Strengths and Weaknesses

The Flawfinder documentation lists, under the section "Reviews/Papers", some strengths and weaknesses of the tool.

- "Low comparative cost and memory cost" [1];
- "Can find vulnerabilities in the categories V1 (Stack-based buffer overflow), V2 (Heap-based buffer overflow), and V4 (Format string vulnerabilities)" [1].
- "When the tool does find an error, it is extremely useful" [8]. Indeed, it provides CWE references. Yet, some warnings are too generic.
- "Applicability limitations A1 (Source code required) and A10 (Only protects libc string manipulation functions)" [1].
- "Protection limitation P17 (False negatives are possible)" [1].
- "There are papers that pointed out practical limitations" [6]. For instance, it does not detect logical vulnerabilities or runtime issues.
- It has a high "False positives" [8].

2 A Use-Case: Analysis of Warnings

The execution of the tool on the proposed code produced the output shown in Figure 1. Below are the key warnings found on the reported lines and their analysis:

8 The problem refers to the eventuality of `src` not being *null-terminated*, thus causing a memory access violation when reading its length with `strlen` and using this amount as another string length.

9-10 The line 9 has the same problem listed above. In this case, there is also the need of null-terminating a string after using `strncpy` and the tool warns us of possible error. Indeed, the index for the last character is determined incorrectly, since the length of the `dest` string is used without subtracting 1 to it. Moreover, this length is calculated *before* adding the null terminator to `dest`.

17, 22 After using `read`, we should check the function return value to verify that the buffer has been correctly filled.

28, 33, 34, 35 These are all false positives. For lines 28 and 33, the tool warns us that we may underestimate the actual size needed to the array, and suggests to determine it dynamically. But `fgtes` ensures that the buffer limit is not exceeded. Also, in lines 33-35, 1024 characters are copied with `strncpy` and 19 are concatenated with `strcat`, leaving 1 character left for the null-terminator. The warning of missing a null-terminator in line 34 is not real because the null-terminator will be added through `strcat` in line 35.

42 This error is real: the use of `strcpy` does not set a limit for the copying operation, and a memory violation can be easily achieved. Moreover, after copying a string into `buffer` the null-terminator character is not added.

56 This basically spotted a string-format vulnerability in the code.

Unfortunately, Flawfinder did *not* flag the following vulnerabilities:

- Line 41: Even if the size of the buffer is not computed statically, here there is a constant 10 that is will not be suitable for many input strings.
- Lines 60-63: The loop should first reduce the `y` value and then use it as index for accessing the array `a`. In this way, the first loop iteration will cause a memory access violation. Moreover, the check should be exclusive: `y > 0`.
- In general, the code misses some check of null arguments.
- In general, the tool doesn't help us with possible memory leaks caused by missing `free` on dynamically allocated memory.

Of course, the tool did not spot the compile errors, that are leaved to the compiler:

- Missing # symbol in the include instructions;

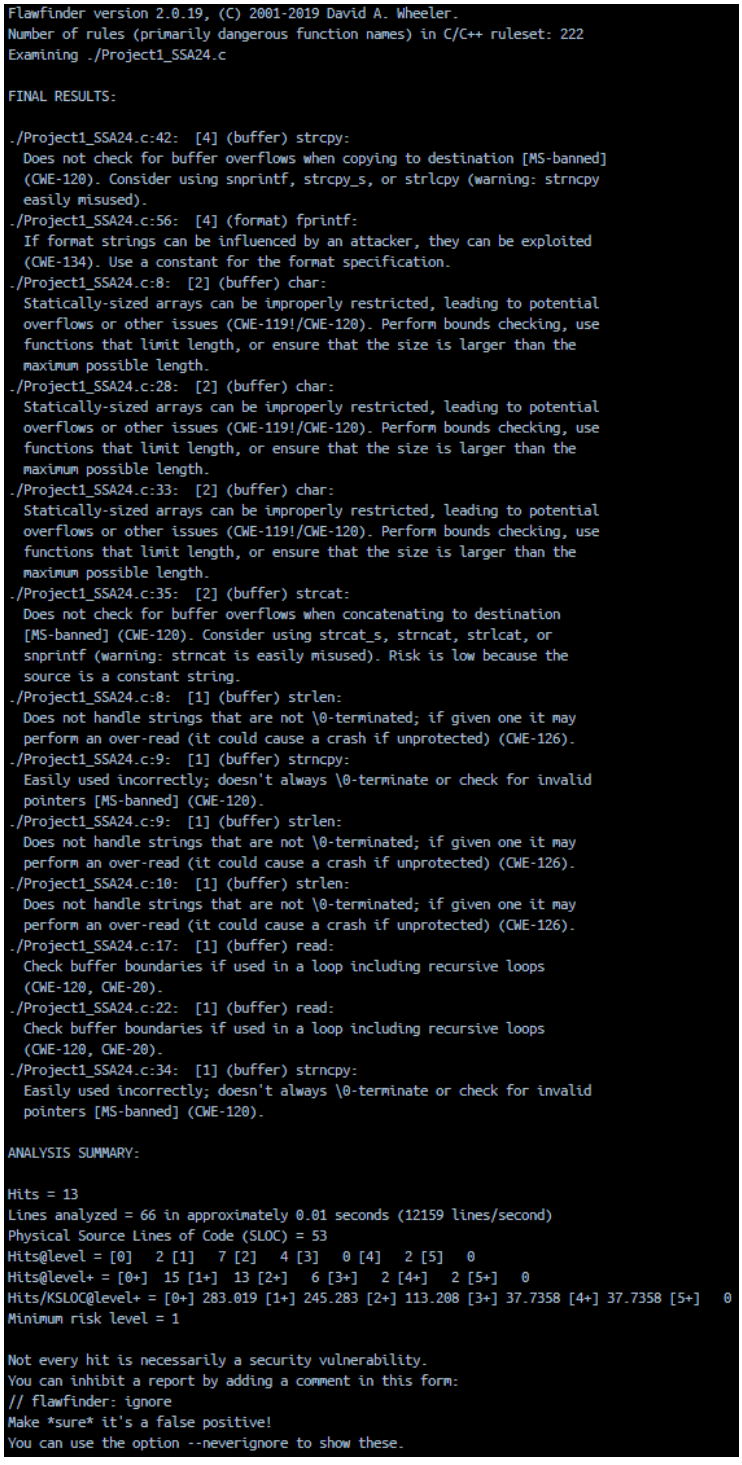

Figure 1: Warnings produces by the tool Flawfinder on the code Projcet1_SSA24.c

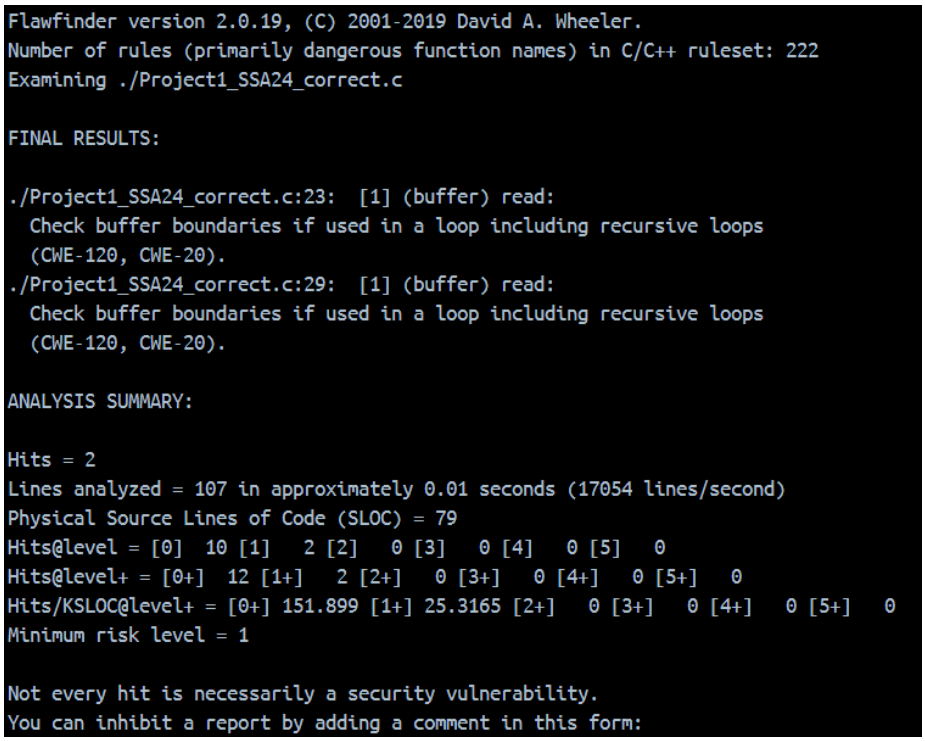

Figure 2: Flawfinder result on the corrected code.

- Invalid try-catch block in function `main`;
- Missing closing parenthesis } for the `main` function;
- Missing semicolon at line 58;
- Undefined term aFile in line 58;
- Missing some headers (e.g., `<ctype.h>` for the `isAlpha` function).

3 Correct Code

Following a series of security-focused code modifications, a re-analysis using Flawfinder was conducted, the results of which are presented in Figure 2. This scan identified two persistent notifications, which, after extensive investigation and thorough research, have been conclusively identified as false positives. Despite considerable efforts and the implementation of various corrective measures, these specific alerts could not be suppressed, indicating an inherent limitation or misinterpretation by the analysis tool in this particular context.

```c
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
#include <ctype.h>

void func1(const char *src) {
    size_t len = strlen(src);
    char *dst = malloc(len + 1);
    if (dst) {
        strncpy(dst, src, len);
        dst[len] = '\0';
        free(dst);
    }
}

void func2(int fd) {
    char *buf;
    size_t len;
    if (read(fd, &len, sizeof(len)) != sizeof(len) || len > 1024)
        return;
    buf = malloc(len + 1);
    if (buf) {
        if (read(fd, buf, len) == len) {
            buf[len] = '\0';
        }
        free(buf);
    }
}

void func3() {
    char buffer[1024];
    printf("Please enter your user id:");
    if (fgets(buffer, sizeof(buffer), stdin)) {
        if (!isalpha(buffer[0])) {
            char errormsg[1044];
            snprintf(errormsg, sizeof(errormsg), "%s
 is not a valid ID", buffer);
        }
    }
}

void func4(const char *foo) {
    char *buffer = (char *)malloc(11);
    if (buffer) {
        strncpy(buffer, foo, 10);
        buffer[10] = '\0';
        free(buffer);
    }
}

int main() {
    func4("safe_input");
    func3();
    return 0;
}
```


Figure 3: The correct version of the code